

PORTFOLIO

직접 굴려보며 이해하고, 반복되는 불편을 도구로 바꿉니다.

개인 서비스를 운영하며 반복하던 배포와 VM 설정을 도구로 만들었습니다. 각 프로젝트에서 해결하려던 문제, 선택한 방법, 직접 구현한 내용과 확인한 결과를 소개합니다.

-
- | | | |
|----|--|-----------------------------|
| 01 | Heimdall
반복 배포 절차를 자동화하고, 새 버전의 실행 실패에도 기존 서비스 유지 | DEPLOYMENT ·
P.03-04 |
| 02 | Gjallar
VM 사양 재사용·IP 사전 확인으로 반복 설정을 줄이고 변경 결과 검증 | INFRASTRUCTURE ·
P.05-06 |
| 03 | K-Le-PaaS
자연어 운영 제어·모니터링과 Git 연동 자동 배포를 제공하는 팀 플랫폼 | KUBERNETES ·
P.07-09 |
-

BACKGROUND

홈랩 운영에서 출발한 인프라 자동화 프로젝트

홈랩에 서비스를 추가할 때마다 VM 사양과 IP를 정하고, 빌드·실행·외부 접근 경로를 준비했습니다. 반복 작업을 줄이는 개인 도구를 만들고, 팀 프로젝트에서는 자연어 요청을 운영 기능에 연결했습니다.

경험	본인 역할과 해결한 범위
Heimdall · 개인	배포 흐름 설계·개발. Git 저장소 빌드부터 서비스 응답 확인, 검증된 버전의 Preview 전환까지 구현.
Gjallar · 개인	VM 운영 도구 설계·개발. 사양 재사용, 생성 전 조건 확인, 승인·실행·변경 후 상태 조회 구현.
K-Le-PaaS · 2인 팀	자연어 요구 변환과 인프라 모니터링 담당. Kubernetes 실행·조회 API와 Prometheus 지표 조회 구현.

HOME LAB

개인 서비스 운영 환경

Proxmox 3노드에서 Linux VM과 서비스를 운영합니다. IPFire로 관리망과 서비스망을 분리하고, NAS/NFS와 WireGuard·OCI를 연결해 스토리지와 외부 접근 경로를 구성했습니다.

Proxmox VE 3-node

IPFire

NAS / NFS

WireGuard / OCI

TECHNOLOGY

Infrastructure

Linux · Proxmox · Docker ·
Kubernetes

Backend

Python · FastAPI · PostgreSQL

Delivery

Git · NGINX · GitHub Actions

Monitoring

Prometheus · REST API

추가 구축 경험: Kolla-Ansible 기반 OpenStack 멀티노드 PoC에서 인스턴스 통신, Floating IP, Cinder 볼륨 연결 확인. 단일 Controller 테스트 환경입니다.

01 HEIMDALL

APPLICATION DELIVERY · PERSONAL PROJECT · 2026.08-PRESENT

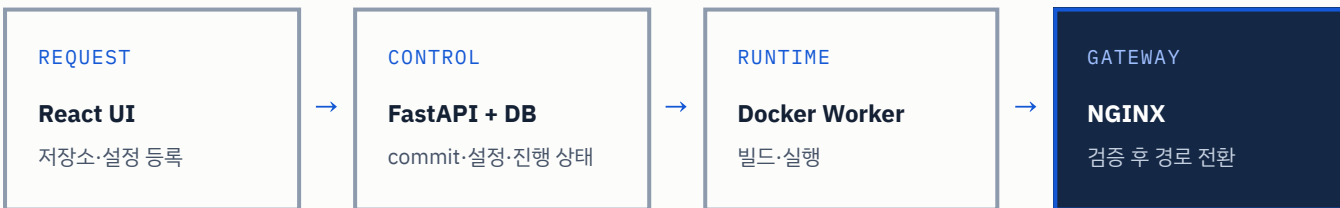
반복 배포 절차의 자동화

개인 서비스를 하나 추가할 때마다 VM과 실행 환경을 준비하고, 저장소 빌드부터 외부 접근 경로까지 수동으로 연결했습니다. 서비스마다 같은 준비를 반복하지 않도록 공용 서버에서 배포를 처리하고, 새 버전의 시작 실패가 기존 서비스에 영향을 주지 않게 만들었습니다.

PROBLEM	PROPOSAL	WHY THIS APPROACH
수작업 배포와 개별 검증 저장소 checkout, Docker build·실행, 환경 설정과 NGINX 경로를 매번 연결했습니다. 실행 결과와 실제 서비스 응답도 따로 확인해야 했습니다.	빌드·검증·공개 자동화 저장소와 설정을 등록하면 새 버전을 별도로 실행하고, 서비스 응답과 공개 경로를 확인한 뒤 현재 버전으로 전환하도록 설계했습니다.	기존 서비스 유지와 후보 검증 새 배포 후보를 기존 컨테이너와 분리했습니다. 검증을 통과한 후보만 활성화하면 시작에 실패한 버전이 기존 서비스 경로를 대체하지 않습니다.

설계 선택 · VM 운영과 애플리케이션 배포 분리

초기에는 VM 생성까지 한 흐름에 넣으려 했지만, 생성 시험 중 향후 MCP가 받은 요구가 VM 생성인지 애플리케이션 배포인지 구분하기 어려워질 수 있다고 판단했습니다. 통합 흐름을 완성하기 전에 VM 운영은 Gjallar로 분리하고, Heimdall은 이미 준비된 서버에 배포하도록 범위를 좁혔습니다.



달라진 배포 방식

공용 서버에 배포

서비스마다 VM을 새로 만들던 방식에서, 실행 중인 서버에 저장소를 등록하고 빌드·검증·공개 경로 전환을 이어가는 방식으로 바꿨습니다.

개인 환경에서 확인

애플리케이션 배포가 **3분 이내**에 완료된 것을 확인했습니다. VM 생성은 포함하지 않으며, 개인 환경에서 확인한 사례입니다.

본인 역할: 개인 프로젝트 설계·개발 · Python / FastAPI / Docker / NGINX / PostgreSQL / React

IMPLEMENTATION · TROUBLESHOOTING

재배포 후 Preview 접속 장애 해결

앱 내부의 Backend·DB 호출과 Gateway 내부 응답은 정상이었습니다. 반면 호스트에서 Preview 포트로 보내는 요청만 timeout돼, 앱 동작과 외부 접근 경로를 나눠 확인했습니다.

진단 · 실패 구간 재현

Control DB와 Docker의 Preview 포트가 일치함을 확인했습니다. Docker Desktop에서 Gateway의 최초 네트워크를 분리한 뒤 호스트 포트만 응답하지 않는 현상을 재현했습니다.

원인 · 검증 시점의 공백

기존 로직은 네트워크를 분리하기 전에 응답을 검사했습니다. 검사 이후 정리 단계에서 접속 경로가 깨져도 배포가 성공으로 확정되는 순서였습니다.

수정 · 전환 후 재검증

새 배포 네트워크에서 Gateway를 동일 포트로 재생성하고, 호스트 응답을 다시 확인한 뒤 활성 배포를 확정하도록 수정했습니다. 재생성·검증 실패 시 이전 설정과 네트워크로 Gateway를 복원합니다.

RESULT · 2026.08.11 검증 기록

연속 배포 시험

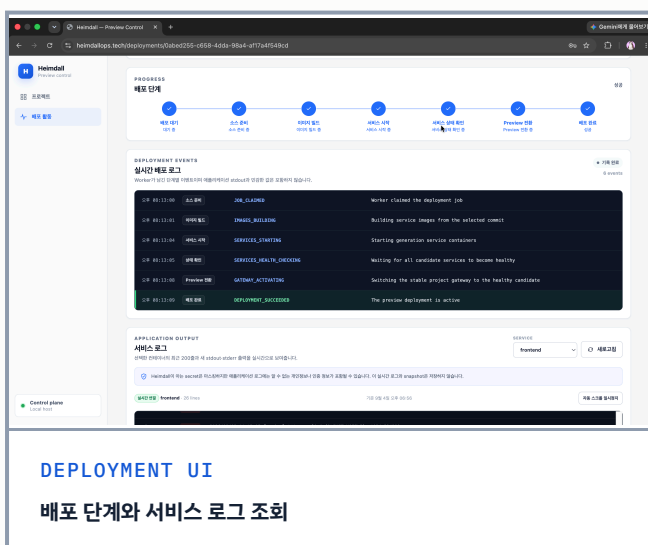
두 번째 배포 뒤에도 같은 Preview 포트에서 새 배포 식별자와 정상 응답을 확인했습니다. Gateway를 정지한 뒤 다음 배포에서 동일 포트로 복구되는 것도 확인했습니다.

실제 앱 재배포

수정한 Worker로 재기동한 뒤 같은 commit을 다시 배포했습니다. Preview 포트 유지, Web-API HTTP 200, managed database 연결 성공을 확인했습니다.

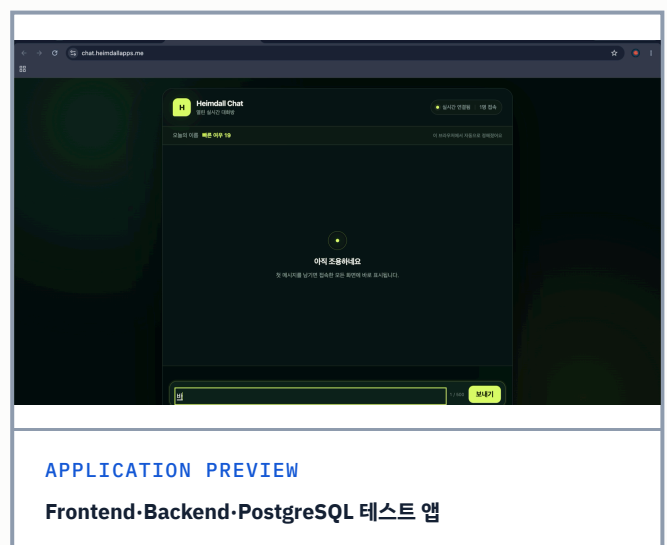
재발 방지

Gateway 네트워크 전환 후 동일 포트 유지와 실패 시 이전 Gateway 복원을 확인하는 회귀 테스트를 추가했습니다.



DEPLOYMENT UI

배포 단계와 서비스 로그 조회



APPLICATION PREVIEW

Frontend·Backend·PostgreSQL 테스트 앱

전환 중 Gateway 재생성이 필요해 짧은 접속 중단이 생길 수 있습니다. 검증 완료 후 배포를 확정하며, 실패 시 기존 서비스로 복구하도록 했습니다.

[2-minute demo](#)[진단·실제 검증 기록](#)[회귀 테스트](#)

02 GJALLAR

INFRASTRUCTURE OPERATIONS · PERSONAL PROJECT · 2026.05-PRESENT

VM 사양 재사용과 생성 전 검증

홈랩에서 같은 용도의 VM을 만들 때도 vCPU·메모리·디스크를 다시 입력했고, 수동으로 주소를 관리하다 IP가 충돌했습니다. 사양을 재사용하고 현재 자원 상태를 확인한 뒤 VM을 생성하는 도구를 만들었습니다.

PROBLEM	PROPOSAL	WHY THIS APPROACH
반복 설정과 수동 IP 관리 VM마다 사양과 주소를 다시 정하는 과정이 번거로웠습니다. 설정값을 입력하는 작업과 실제 자원·주소 상태를 확인하는 작업도 따로 수행했습니다.	사양 재사용과 조건 검증 템플릿 위에 VM Profile을 두어 사양을 재사용합니다. IP·네트워크·스토리지 조건은 Proxmox에서 조회해 생성 요청 전에 확인하도록 했습니다.	Proxmox API 기반 상태 확인 기존 도구를 조합한 자동화에서 설정과 상태 관리가 늘어났습니다. 운영 부담을 줄이기 위해 Proxmox가 제공하는 조회·변경 API를 바탕으로 필요한 기능을 직접 구현했습니다.

실제 자원 상태 Proxmox VE 노드·VM·스토리지·네트워크 비동기 작업 상태 변경 후 실제 VM 상태	조회 · 사전 확인 ↔ 승인된 변경 요청	요청과 실행 관리 Gjallar VM Profile·생성 계획 권한·승인·중복 방지 작업 추적·결과 기록
---	--	--

추가 설계 · 요청 접수와 완료 판단 분리

Proxmox의 변경 작업은 비동기로 진행됩니다. 요청 응답을 받는 것만으로 완료를 판단하지 않고, 작업 종료와 실제 VM 상태를 다시 확인하도록 했습니다.

본인 역할: 개인 프로젝트 설계·개발 · Python / FastAPI / PostgreSQL / Proxmox API / React

IMPLEMENTATION · TROUBLESHOOTING

디스크 크기 사전 검증과 VM 생성 오류 예방

초기 Terraform 기반 생성 시험에서 200GB 템플릿에 50GB 디스크를 요청했습니다. VM 복제는 완료됐지만 디스크 조정이 실패해 정지된 VM이 남았습니다.

01 READ 선택한 템플릿의 실제 디스크 조회	02 COMPARE 요청 크기와 템플릿 크기 비교	03 PREFLIGHT 작은 요청 차단 · 유효한 계획만 생성
---	--	---

원인 확인	실행 로그의 requested size is lower than current size와 템플릿 설정을 대조했습니다. 요청이 기존 디스크의 축소를 요구했지만 사전 검증에서 이 조건을 확인하지 않았습니다.
사전 검사 추가	템플릿 설정에서 실제 디스크 크기를 읽고 cloud-init-CD-ROM 장치는 제외했습니다. 요청 크기가 템플릿보다 작으면 생성 단계로 넘어가지 않도록 차단했습니다.
재발 방지	Proxmox API 기반으로 생성 경로를 전환한 뒤에도 템플릿 디스크 크기 비교와 작은 요청을 거부하는 회귀 테스트를 유지합니다.

RESULT · 2026.05.03 실제 시험

실패 조건 재입력	200GB 템플릿에 50GB를 요청하면 사전 검증에서 차단했습니다. 복제 이후에야 발견하던 제약을 생성 전 단계로 옮겼습니다.
유효한 요청 실행	200GB 요청에서는 실제 VM 생성이 완료됐습니다. 시험 후 VM을 정상 종료하고 정지 상태를 확인했습니다.

위 실패와 개선 검증은 초기 Terraform 경로의 기록입니다. 이후 Proxmox API 기반으로 생성 경로를 전환했습니다.

현재 생성 경로 검증 · 2026.06.01 홈랩 시험

생성·기동 작업	clone-start 작업의 성공과 실제 VM의 running 상태를 확인했습니다.
초기 설정 완료	guest-agent 응답, 요청한 IP 설정, cloud-init 완료를 확인했습니다. 요청 접수뿐 아니라 VM이 준비됐는지까지 확인했습니다.

[실패·성공 시험 기록 ↗](#)[현재 회귀 테스트 ↗](#)

03 K-LE-PAAS

KUBERNETES OPERATIONS · TEAM OF 2 · 2025.09-2025.12

자연어 운영 제어와 Git 연동 배포

서비스를 운영하려면 여러 도구를 오가며 명령과 절차를 알아야 했습니다. 이 부담을 줄이기 위해 웹·Slack에서 자연어로 운영을 요청하고, Git의 코드 변경은 자동으로 배포되는 플랫폼을 2인 팀으로 만들었습니다.

PROBLEM	PROPOSAL	WHY THIS APPROACH
도구별로 분산된 운영 절차 재시작·상태 조회에는 Kubernetes 명령이 필요했고, 배포 진행 상황과 자원 상태도 각각 확인해야 했습니다. 작업을 수행하려면 도구별 사용법과 절차를 알아야 했습니다.	자연어 제어와 자동 배포 재시작·상태 조회·파드 수 조절을 자연어 요청에 연결했습니다. 팀에서 Git 연동 배포를 구성하고, 운영 결과와 자원 지표를 화면에서 확인할 수 있게 했습니다.	실행 계획 검증과 작업 제한 자연어 해석 오류를 고려해 자유 형식 명령을 바로 실행하지 않도록 했습니다. CommandRequest와 허용된 CommandPlan으로 실행 형식과 지원 작업을 제한했습니다.



담당 범위

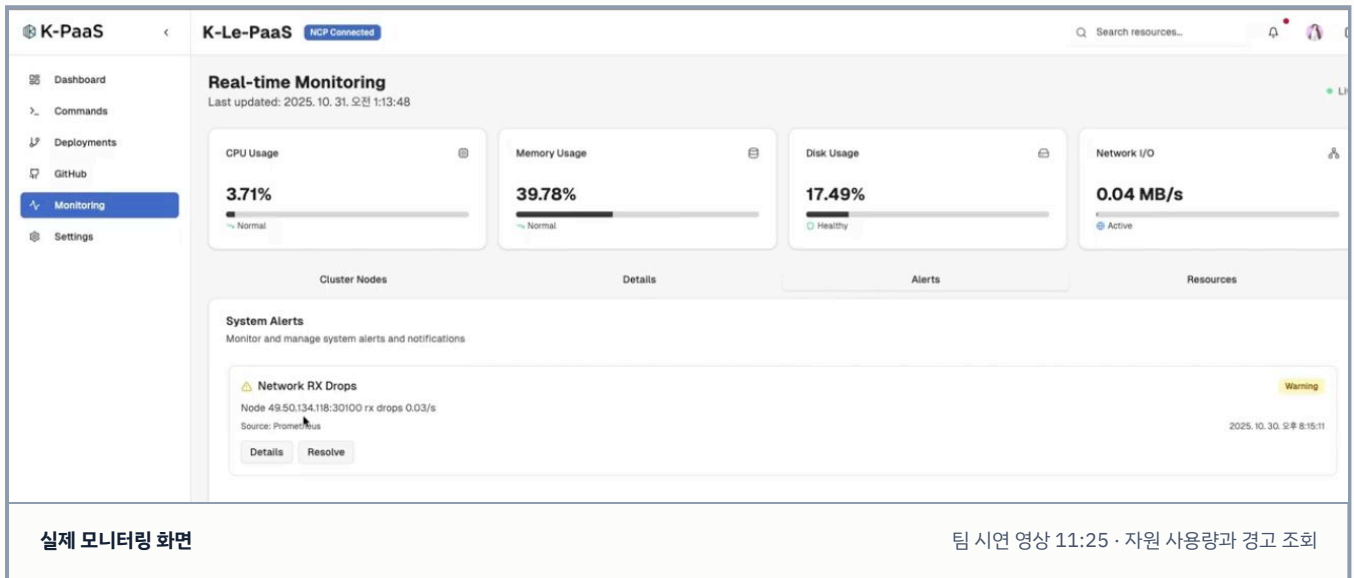
저는 자연어 변환과 상태·로그 조회, 재시작·파드 수 조절 API를 담당했습니다. 모니터링은 Prometheus 설치·수집 대상 구성부터 지표 조회 API와 대시보드 연결까지 맡았습니다.

Python / FastAPI / Kubernetes / Prometheus / NCP / Argo CD

IMPLEMENTATION · MONITORING

자원 지표 수집과 모니터링 기능 구현

Prometheus 설치와 수집 대상을 구성하고, CPU·메모리·디스크·네트워크 지표를 조회 API와 화면에 연결했습니다. 자연어 운영 요청과 함께 자원 사용량과 상태를 확인할 수 있게 했습니다.



수집 · 조회 · 표시 구현 범위

지표 수집 구성

Prometheus 설치와 수집 대상 구성을 맡았습니다. CPU·메모리·디스크·네트워크 지표를 조회할 수 있도록 연결했습니다.

조회 API 구현

Prometheus 질의 결과를 화면에서 사용할 수 있는 API로 제공했습니다. 사용량뿐 아니라 CPU I/O wait, 디스크 상태, 네트워크 송수신 등 상세 지표 조회도 구현했습니다.

대시보드 연결

REST API를 10초 간격으로 조회해 지표를 갱신합니다. 사용자는 명령을 따로 입력하지 않고 자원 사용량을 확인할 수 있습니다.

운영 제어와 모니터링 연계

자연어 제어는 요청을 실제 운영 작업에 연결하고, 모니터링은 실행 중인 자원의 상태를 확인하는 기능입니다. 두 기능을 같은 플랫폼에서 사용할 수 있도록 구성했습니다.

[모니터링 시연](#) ➤ [Backend](#) ➤ [Frontend](#) ➤

화면에 표시된 사용량은 시연 당시의 값입니다. 운영 성능 개선율이나 장애 감소 수치로 사용하지 않습니다.

RESULT · TEAM DEMO

자동 배포 시연과 실제 동작 검증

팀에서 구성한 Git 연동 배포는 코드 변경을 반영하면 빌드와 배포로 이어집니다. 서로 다른 두 애플리케이션의 시연에서 배포 완료 상태와 소요 시간을 확인했습니다.

01	02	03
CODE	BUILD	DEPLOY
PR 병합 또는 코드 변경 push	변경한 소스로 이미지 빌드	배포 실행 · 완료 결과 확인

시연 기록 · 배포 소요 시간

시연 배포	소요 시간	화면에서 확인한 결과
test01 · 배포 #2 GitHub PR 병합으로 시작	1분 58초	Success · Duration 1m 58s 시연 01:51 ↗
test03 · 배포 #4 코드 수정 후 push로 시작	4분 30초	Success · Duration 4m 30s 시연 04:23 ↗

서비스 접속과 자연어 제어 검증

배포 후 서비스 접속	첫 번째 배포가 완료된 뒤 test01 도메인에 접속해 응답 본문을 확인했습니다. 배포 완료 표시와 실제 서비스 접근을 함께 확인한 장면입니다. 시연 02:30 ↗
자연어 요청의 반영	파드 수 조절을 자연어로 요청하고, 실행 뒤 터미널의 파드 목록에서 요청한 상태로 변경된 것을 확인했습니다. 시연 09:31부터 ↗

담당 범위와 기록 기준

개인 담당은 자연어 제어와 모니터링입니다. 위 배포 시간은 팀이 완성한 플랫폼의 시연 결과이며, 평균·보장 시간이나 수동 대비 단축률을 뜻하지 않습니다. 소요 시간은 영상 재생 시간이 아닌 배포 화면의 Duration 값을 기준으로 했습니다.

[전체 시연 ↗](#) [Backend ↗](#) [Frontend ↗](#)